

WEEK 5:

Simulated Annealing to solve 8-Queens problem

```
import random
import math

N = 8

# Count number of attacking pairs
def conflicts(board):
    count = 0

    for i in range(N):
        for j in range(i + 1, N):

            # Same diagonal
            if abs(board[i] - board[j]) == abs(i - j):
                count += 1

    return count

def simulated_annealing():

    # Random initial solution
    board = [random.randint(0, N - 1) for _ in range(N)]

    current_cost = conflicts(board)
```

```
print("Initial board:", board)
print("Initial conflicts:", current_cost)

temperature = 100

while temperature > 0.001:

    # Solution found
    if current_cost == 0:
        return board

    # Create neighboring solution
    new_board = board.copy()

    row = random.randint(0, N - 1)
    new_board[row] = random.randint(0, N - 1)

    new_cost = conflicts(new_board)

    delta = new_cost - current_cost

    # Accept better solution
    if delta < 0:
        board = new_board
        current_cost = new_cost
```

```
# Accept worse solution with probability
elif random.random() < math.exp(-delta / temperature):
    board = new_board
    current_cost = new_cost

    temperature *= 0.99

return board

# Display board
def print_board(board):

    for row in range(N):
        for col in range(N):

            if board[row] == col:
                print("Q", end=" ")
            else:
                print(".", end=" ")

        print()
```

```
# Main program
solution = simulated_annealing()

print("\nFinal Board:")
print_board(solution)

print("\nFinal conflicts:", conflicts(solution))

if conflicts(solution) == 0:
    print("Solution found!")
else:
    print("No solution found.")
```

OUTPUT:

```
... Initial board: [0, 6, 0, 6, 2, 2, 4, 5]
Initial conflicts: 7
```

```
Final Board:
```

```
. . Q . . . . .
. . . . . Q .
. . . Q . . . .
Q . . . . . .
. . . . Q . . .
. Q . . . . . .
. . . . Q . . .
. . . . . Q .
```

```
Final conflicts: 0
Solution found!
```